KARTHIKEYAN T

SQL That Does the Work

The 20% of SQL behind 80% of real queries, with the traps that catch people out.



11
core patterns

8
reusable recipes

12
exercises with answers

1
page cheat sheet

Every query run and verified on PostgreSQL

carthworks.vercel.app

CONTENTS

What's inside

THE CORE PATTERNS

01 Filter & Aggregate 5

02 Joins 6

03 CTEs 9

04 Window Functions 11

05 Subqueries 13

06 Conditional Logic 15

07 Dates & Strings 17

08 Set Operations 19

09 NULLs & Counting 20

10 Sorting & Limiting 22

11 Beyond SELECT 24

PUT IT TO WORK

· 8 queries you will reuse 26

· 10 silent bugs to check before you ship 31

· 12 exercises 32

· Answers to the exercises 33

· 8 questions, answered in one breath 36

REFERENCE

· One-page cheat sheet 37

· Moving between databases 38

· Sample database script 39

· About the author 40

START HERE

Start here

A small set of SQL covers most real work. This guide is that set.

You don't need every feature of SQL to be useful at work. You need the handful of patterns that come up daily, and you need to know where they silently go wrong. Each chapter gives you the pattern, a query you can run, its actual result, and the mistakes worth avoiding.

How to use this guide

1. **Load the sample database** using the script in the appendix, so you can run everything.
2. **Change each query** before moving on. Predict the result, then check it.
3. **Do the 12 exercises** without looking, then compare answers.
4. **Keep the cheat sheet and checklist** beside you at work.

The idea that explains most SQL errors

SQL runs in a different order from how you write it. That's why `WHERE` can't see a `SELECT` alias, and why window functions can't go in `WHERE`.

FROM JOIN	WHERE	GROUP BY	HAVING	Window functions	SELECT	ORDER BY	LIMIT

The sample database

A small fictional electronics shop, plus its staff. Every result in this guide comes from these six tables.

`customers`
id, name, email*, city*, signup_date

`orders`
id, customer_id*, order_date, status, amount

`order_items`
order_id, product_id, qty

`products`
id, name, category, price

`employees`
id, name, dept, salary, manager_id*

`signups_raw`
id, email, source, created_at

* Can be NULL, on purpose. `customer_id` is NULL for a guest order, which sets up some of the traps ahead.

THE MAP

SQL on one page

Five sublanguages, plus the building blocks they share. Chapters 1–10 live almost entirely in the first box.

DQL Read data — Chapters 1–10

`SELECT · FROM · WHERE · GROUP BY · HAVING · ORDER BY · LIMIT`

`Joins: INNER · LEFT · RIGHT · FULL OUTER · CROSS`

DML Change rows — Chapter 11

`INSERT · UPDATE · DELETE`

DDL Define structure — Chapter 11

`CREATE · ALTER · DROP · TRUNCATE`, for tables, views, indexes, and constraints

TCL Group changes — Chapter 11

`BEGIN · COMMIT · ROLLBACK`

DCL Control access — Chapter 11

`GRANT · REVOKE`

The building blocks

Operators

Comparison: `= <> > <`

Logical: `AND OR NOT`

Membership: `IN BETWEEN LIKE`

Null test: `IS NULL`

Functions

Numeric: `ROUND ABS`

String: `LOWER TRIM CONCAT`

Date: `DATE_TRUNC EXTRACT`

Null: `COALESCE NULLIF`

Aggregate: `COUNT SUM AVG`

Window: `RANK LAG`

Data types

Numeric: `INT NUMERIC`

String: `TEXT VARCHAR`

Date & time: `DATE TIMESTAMP`

Boolean: `BOOLEAN`

JSON: `JSONB`

Where to spend your time

Learn DQL deeply first; it is most of what analysts and many developers write each day. Learn the rest well enough to change data without fear.

CHAPTER 01

Filter & Aggregate

Answer the question the business actually asked.

Most reporting queries have the same shape: pick the rows, group them, compute a number, and keep the groups that matter. Learn the difference between the two filters and you have the shape.

WHERE filters rows. HAVING filters groups.

```sql
SELECT customer_id,
       COUNT(*)    AS orders,
       SUM(amount) AS revenue
FROM orders
WHERE status = 'completed'      -- row filter: runs before grouping
  AND customer_id IS NOT NULL
GROUP BY customer_id
HAVING SUM(amount) > 10000      -- group filter: runs after grouping
ORDER BY revenue DESC;
```

RESULT · 3 ROWS

customer_id	orders	revenue
5	2	35500.00
1	4	33800.00
2	3	22200.00

Trap · Mixing AND with OR

`AND` binds tighter than `OR`. `WHERE status = 'pending' OR status = 'cancelled' AND amount > 5000` keeps every pending order, whatever its amount. Use parentheses, or `status IN ('pending','cancelled')`.

Rule of thumb

If a condition could run on a single row, put it in `WHERE`. The database then groups fewer rows, and the intent is clearer.

DISTINCT is not a fix

```sql
SELECT COUNT(*)                    AS orders,
       COUNT(DISTINCT customer_id) AS unique_buyers
FROM orders
WHERE status = 'completed';
```

RESULT · 1 ROW

orders	unique_buyers
13	6

`COUNT(DISTINCT ...)` answers a real question. `SELECT DISTINCT` added because the results "look duplicated" usually hides a join problem. Find the cause, covered in the next chapter.

CHAPTER 02

Joins

Combine tables without losing rows, or inventing them.

Before you write a join, say out loud what one output row represents. Most join bugs come from skipping that sentence.

INNER keeps matches. LEFT keeps everything on the left.

```sql
SELECT c.name, o.id AS order_id, o.amount
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE c.city = 'Chennai'
ORDER BY c.name, o.id;
```

RESULT · 7 ROWS

name	order_id	amount
Asha Rao	101	5700.00
Asha Rao	103	3600.00
Asha Rao	109	6500.00
Asha Rao	115	18000.00
Chitra Nair	104	1200.00
Chitra Nair	110	4500.00
Gita Menon	*NULL*	*NULL*

Gita has no orders. With INNER JOIN she would vanish; with LEFT JOIN she stays, with NULLs.

The anti-join: rows with no match

```sql
SELECT c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

RESULT · 1 ROW

name
Gita Menon

Trap · A WHERE clause can silently undo a LEFT JOIN
Filtering the right table in `WHERE` throws away the NULL rows, so the query behaves like an INNER JOIN. Move the condition into `ON`.

```sql
-- Wrong: Gita disappears, because her NULL status fails the filter
-- ... LEFT JOIN orders o ON o.customer_id = c.id WHERE o.status = 'completed'

-- Right: everyone stays, and only completed orders are counted
SELECT c.name, COUNT(o.id) AS completed_orders
FROM customers c
LEFT JOIN orders o
       ON o.customer_id = c.id AND o.status = 'completed'
GROUP BY c.name
ORDER BY completed_orders DESC, c.name;
```

RESULT · 7 ROWS

name	completed_orders
Asha Rao	4
Ben Thomas	3
Esha Khan	2
Chitra Nair	1
Dev Patel	1
Farid Ali	1
Gita Menon	0

Trap · Fan-out: the join that inflates your revenue
Joining one order to its many items repeats the order's `amount` once per item. The total looks plausible and is wrong.

```sql
SELECT
  (SELECT SUM(amount) FROM orders)  AS true_total,
  (SELECT SUM(o.amount)
   FROM orders o
   JOIN order_items i ON i.order_id = o.id) AS inflated_total;
```

RESULT · 1 ROW

true_total	inflated_total
128300.00	167800.00

The fix: aggregate the “many” side first, then join, so each order is still one row.

```sql
WITH item_counts AS (
  SELECT order_id, SUM(qty) AS units
  FROM order_items
  GROUP BY order_id
)
SELECT SUM(o.amount) AS revenue, SUM(ic.units) AS units
FROM orders o
JOIN item_counts ic ON ic.order_id = o.id;
```

RESULT · 1 ROW

revenue	units
128300.00	22

Self join: a table that refers to itself

```sql
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.id = e.manager_id
ORDER BY e.id
LIMIT 4;
```

RESULT · 4 ROWS

employee	manager
Meera Iyer	*NULL*
Arjun Das	Meera Iyer
Neha Joshi	Meera Iyer
Ravi Kumar	Arjun Das

> **Other join types**
> `FULL JOIN` keeps unmatched rows from both sides, which is handy for reconciling two sources. `CROSS JOIN` produces every combination; use it deliberately, such as dates × products.

CHAPTER 03

CTEs

Break a hard query into steps you can read.

A common table expression (CTE) names an intermediate result. Think of each one as a step you can test alone: run it, check the rows, then build the next step on it.

```sql
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date)::date AS month,
         SUM(amount) AS revenue
  FROM orders
  WHERE status = 'completed'
  GROUP BY 1
),
big_months AS (
  SELECT month, revenue
  FROM monthly
  WHERE revenue > 15000
)
SELECT month, revenue
FROM big_months
ORDER BY month;
```

RESULT · 4 ROWS

month	revenue
2025-01-01	23700.00
2025-03-01	33200.00
2025-04-01	20500.00
2025-07-01	18000.00

Recursive CTEs: walk a hierarchy

An anchor query gives the starting rows. The recursive part runs repeatedly, joining to the previous step, until it returns nothing.

```sql
WITH RECURSIVE org AS (
  SELECT id, name, 1 AS level
  FROM employees
  WHERE manager_id IS NULL      -- anchor: the top of the tree
  UNION ALL
  SELECT e.id, e.name, org.level + 1
  FROM employees e
  JOIN org ON e.manager_id = org.id   -- step: direct reports
)
SELECT level, name FROM org ORDER BY level, name;
```

RESULT · 8 ROWS

level	name
1	Meera Iyer
2	Arjun Das
2	Neha Joshi
2	Sana Sheikh
3	Ravi Kumar
3	Tom Mathew
3	Uma Pillai
3	Vikram Shah

When to use which

Use a **CTE** for readability within a query. Use a **view** when many queries need the same logic. Use a **temp table** when an expensive intermediate result is reused or needs its own index.

CHAPTER 04

Window Functions

Calculate across rows without collapsing them.

`GROUP BY` turns many rows into one. A window function keeps every row and adds a calculated column beside it. `PARTITION BY` sets the group; `ORDER BY` sets the sequence inside it.

Three ways to rank

```sql
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK()       OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense
FROM employees
WHERE dept = 'Engineering';
```

RESULT · 4 ROWS

name	salary	row_num	rnk	dense
Meera Iyer	240000	1	1	1
Arjun Das	180000	2	2	2
Neha Joshi	180000	3	2	2
Ravi Kumar	120000	4	4	3

Arjun and Neha tie. ROW_NUMBER breaks the tie arbitrarily, RANK leaves a gap afterwards, DENSE_RANK does not.

LAG: compare with the previous row

```sql
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date)::date AS month,
         SUM(amount) AS revenue
  FROM orders
  WHERE status = 'completed'
  GROUP BY 1
)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / LAG(revenue) OVER (ORDER BY month), 1) AS growth_pct
FROM monthly
ORDER BY month;
```

RESULT · 6 ROWS

month	revenue	prev	growth_pct
2025-01-01	23700.00	*NULL*	*NULL*
2025-02-01	11900.00	23700.00	-49.8
2025-03-01	33200.00	11900.00	179.0
2025-04-01	20500.00	33200.00	-38.3
2025-05-01	1800.00	20500.00	-91.2
2025-07-01	18000.00	1800.00	900.0

June has no orders, so LAG compares July with May. Recipe 4 shows how to fill such gaps.

Running totals and share of a group

```sql
SELECT id, order_date, amount,
       SUM(amount) OVER (ORDER BY order_date, id) AS running_total
FROM orders
WHERE status = 'completed'
ORDER BY order_date, id
LIMIT 5;
```

RESULT · 5 ROWS

id	order_date	amount	running_total
101	2025-01-15	5700.00	5700.00
102	2025-01-28	18000.00	23700.00
103	2025-02-10	3600.00	27300.00
105	2025-02-25	8300.00	35600.00
106	2025-03-03	2400.00	38000.00

```sql
SELECT name, dept, salary,
       ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY dept), 1) AS pct_of_dept
FROM employees
WHERE dept = 'Sales'
ORDER BY salary DESC;
```

RESULT · 3 ROWS

name	dept	salary	pct_of_dept
Uma Pillai	Sales	160000	39.5
Sana Sheikh	Sales	150000	37.0
Tom Mathew	Sales	95000	23.5

Trap · The default frame treats ties as one

With `ORDER BY` alone, the frame defaults to `RANGE`, so tied rows share one running total. Add a unique tie-breaker, or state `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

```sql
SELECT name, salary,
       SUM(salary) OVER (ORDER BY salary DESC) AS range_default,
       SUM(salary) OVER (ORDER BY salary DESC
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_frame
FROM employees
WHERE dept = 'Engineering';
```

RESULT · 4 ROWS

name	salary	range_default	rows_frame
Meera Iyer	240000	240000	240000
Arjun Das	180000	600000	420000
Neha Joshi	180000	600000	600000
Ravi Kumar	120000	720000	720000

Trap · You cannot filter on a window function in WHERE

`WHERE` runs before window functions exist. Compute the window in a CTE or subquery, then filter outside it. Recipe 1 shows the pattern.

CHAPTER 05

Subqueries

Use one query's answer inside another.

Scalar: a single value

```sql
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

RESULT · 4 ROWS

name	salary
Meera Iyer	240000
Arjun Das	180000
Neha Joshi	180000
Uma Pillai	160000

Correlated: re-evaluated for each outer row

```sql
SELECT e.name, e.dept, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary)
                  FROM employees
                  WHERE dept = e.dept)
ORDER BY e.dept, e.salary DESC;
```

RESULT · 3 ROWS

name	dept	salary
Meera Iyer	Engineering	240000
Uma Pillai	Sales	160000
Sana Sheikh	Sales	150000

“Who earns more than their own department's average?”

EXISTS: does a matching row exist?

```
SELECT c.name
FROM customers c
WHERE EXISTS (SELECT 1
              FROM orders o
              WHERE o.customer_id = c.id
                AND o.amount > 15000)
ORDER BY c.name;
```

RESULT · 4 ROWS

name
Asha Rao
Ben Thomas
Dev Patel
Esha Khan

Trap · NOT IN breaks when the list contains a NULL
One guest order has a NULL `customer_id`. `x NOT IN (..., NULL)` can never be true, so this query returns **no rows at all**, with no error.

```
SELECT name FROM customers
WHERE id NOT IN (SELECT customer_id FROM orders);
```

RESULT · 0 ROWS

```
(0 rows)
```

Zero rows. Gita has no orders, yet she is missing.

```
SELECT c.name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o
                  WHERE o.customer_id = c.id);
```

RESULT · 1 ROW

name
Gita Menon

NOT EXISTS gets it right. Make it your default.

CHAPTER 06

Conditional Logic

Handle edge cases before they handle you.

CASE: turn values into categories

```sql
SELECT id, amount,
       CASE WHEN amount >= 15000 THEN 'large'
            WHEN amount >=  5000 THEN 'medium'
            ELSE 'small'
       END AS size
FROM orders
ORDER BY id
LIMIT 5;
```

RESULT · 5 ROWS

id	amount	size
101	5700.00	medium
102	18000.00	large
103	3600.00	small
104	1200.00	small
105	8300.00	medium

Branches are checked in order, and the first match wins.

Conditional aggregation: a pivot in one query

```sql
SELECT DATE_TRUNC('month', order_date)::date AS month,
       COUNT(*) FILTER (WHERE status = 'completed') AS completed,
       SUM(CASE WHEN status <> 'completed' THEN 1 ELSE 0 END) AS other
FROM orders
GROUP BY 1
ORDER BY 1;
```

RESULT · 6 ROWS

month	completed	other
2025-01-01	2	0
2025-02-01	2	1
2025-03-01	4	0
2025-04-01	3	1
2025-05-01	1	0
2025-07-01	1	0

FILTER is the PostgreSQL form. SUM(CASE ...) works in every major database.

COALESCE and NULLIF

```sql
SELECT name,
       COALESCE(email, 'no email') AS contact,
       COALESCE(city, 'Unknown')   AS city
FROM customers
WHERE email IS NULL OR city IS NULL;
```

RESULT · 2 ROWS

name	contact	city
Chitra Nair	no email	Chennai
Farid Ali	farid@example.com	Unknown

```sql
-- Dividing by zero raises an error. NULLIF turns the 0 into NULL,
-- so months with no cancellations return NULL instead.
SELECT DATE_TRUNC('month', order_date)::date AS month,
       COUNT(*) FILTER (WHERE status = 'completed')
         / NULLIF(COUNT(*) FILTER (WHERE status = 'cancelled'), 0)
         AS completed_per_cancelled
FROM orders
GROUP BY 1
ORDER BY 1;
```

RESULT · 6 ROWS

month	completed_per_cancelled
2025-01-01	*NULL*
2025-02-01	2
2025-03-01	*NULL*
2025-04-01	*NULL*
2025-05-01	*NULL*
2025-07-01	*NULL*

CHAPTER 07

Dates & Strings

The unglamorous work every pipeline runs on.

Group by period

```sql
SELECT DATE_TRUNC('quarter', order_date)::date AS quarter,
       COUNT(*) AS orders
FROM orders
GROUP BY 1
ORDER BY 1;
```

RESULT · 3 ROWS

quarter	orders
2025-01-01	9
2025-04-01	5
2025-07-01	1

Differences and arithmetic

```sql
SELECT c.name,
       MIN(o.order_date) - c.signup_date AS days_to_first_order,
       MIN(o.order_date) + INTERVAL '30 days' AS followup_at
FROM customers c
JOIN orders o ON o.customer_id = c.id
GROUP BY c.name, c.signup_date
ORDER BY days_to_first_order
LIMIT 3;
```

RESULT · 3 ROWS

name	days_to_first_order	followup_at
Asha Rao	3	2025-02-14 00:00:00
Esha Khan	6	2025-04-10 00:00:00
Dev Patel	8	2025-03-27 00:00:00

Trap · BETWEEN on timestamps misses most of the last day

`created_at BETWEEN '2025-01-01' AND '2025-01-31'` stops at midnight when January 31 begins. Use a half-open range instead. It also never double-counts a boundary.

```sql
SELECT COUNT(*) AS jan_signups
FROM signups_raw
WHERE created_at >= '2025-01-01'
  AND created_at <  '2025-02-01';
```

RESULT · 1 ROW

jan_signups
3

Keep your date column bare
`WHERE DATE_TRUNC('month', d) = ...` usually stops a plain index on `d` from being used. A range condition on the raw column can use it.

Clean up strings

```sql
SELECT email AS raw,
       LOWER(TRIM(email)) AS clean,
       SPLIT_PART(LOWER(TRIM(email)), '@', 1) AS username
FROM signups_raw
WHERE email ILIKE 'asha%';
```

RESULT · 2 ROWS

raw	clean	username
asha@example.com	asha@example.com	asha
ASHA@example.com	asha@example.com	asha

`ILIKE` is PostgreSQL's case-insensitive match; elsewhere, write `LOWER(col) LIKE 'asha%'`. A pattern with a leading wildcard (`'%asha'`) generally can't use a normal index.

CHAPTER 08

Set Operations

Stack, compare, and subtract result sets.

Joins add columns. Set operations add or remove rows. Both queries must return the same number of columns, with compatible types.

```sql
SELECT customer_id FROM orders
WHERE order_date < '2025-04-01' AND customer_id IS NOT NULL
INTERSECT
SELECT customer_id FROM orders
WHERE order_date >= '2025-04-01'
ORDER BY customer_id;
```

RESULT · 5 ROWS

customer_id
1
2
3
4
5

Customers who ordered both before and after April 1.

```sql
SELECT id FROM customers
EXCEPT
SELECT customer_id FROM orders;
```

RESULT · 1 ROW

id
7

Customers with no orders. Oracle traditionally spells this MINUS.

UNION or UNION ALL?

`UNION` removes duplicates, which costs a sort or hash and may delete rows you meant to keep. Default to `UNION ALL`, unless you truly want deduplication.

```sql
SELECT 'union' AS kind, COUNT(*) AS rows_returned
FROM (SELECT city FROM customers UNION SELECT city FROM customers) t
UNION ALL
SELECT 'union all', COUNT(*)
FROM (SELECT city FROM customers UNION ALL SELECT city FROM customers) t;
```

RESULT · 2 ROWS

kind	rows_returned
union	4
union all	14

CHAPTER 09

NULLs & Counting

Count what matters, and handle what's missing.

NULL means “unknown”, not zero and not empty. Any comparison with it returns unknown, and `WHERE` keeps only rows that are true.

Expression	Result	Why
`NULL = NULL`	unknown	Two unknowns aren't known to match
`col = NULL`	unknown	Always wrong; write `col IS NULL`
`NULL + 100`	NULL	Arithmetic spreads NULL
`TRUE OR NULL`	true	True whatever the unknown is
`FALSE AND NULL`	false	False whatever the unknown is

Three counts that look alike

```
SELECT COUNT(*)              AS all_rows,
       COUNT(email)          AS with_email,
       COUNT(DISTINCT city) AS distinct_cities
FROM customers;
```

RESULT · 1 ROW

all_rows	with_email	distinct_cities
7	6	3

COUNT(*) counts rows. COUNT(col) skips NULLs. DISTINCT also skips NULLs.

Trap · <> quietly drops NULL rows

“Everyone outside Chennai” should include Farid, whose city is unknown. This version does not.

```
SELECT name, city
FROM customers
WHERE city <> 'Chennai'
ORDER BY name;
```

RESULT · 3 ROWS

name	city
Ben Thomas	Bengaluru
Dev Patel	Mumbai
Esha Khan	Bengaluru

```sql
SELECT name, city
FROM customers
WHERE city IS DISTINCT FROM 'Chennai'
ORDER BY name;
```

RESULT · 4 ROWS

name	city
Ben Thomas	Bengaluru
Dev Patel	Mumbai
Esha Khan	Bengaluru
Farid Ali	*NULL*

IS DISTINCT FROM is NULL-safe. Elsewhere, add OR city IS NULL.

Trap · AVG ignores NULLs, which may not be what you mean

If NULL really means zero, say so. `AVG(COALESCE(x, 0))` and `AVG(x)` answer different questions. Decide which one the business is asking.

CHAPTER 10

Sorting & Limiting

Control what comes back, and in what order.

Without `ORDER BY`, row order is not guaranteed, even if it looks stable today. With `LIMIT`, that turns into wrong answers.

```sql
SELECT name, city
FROM customers
ORDER BY city NULLS LAST, name
LIMIT 4;
```

RESULT · 4 ROWS

name	city
Ben Thomas	Bengaluru
Esha Khan	Bengaluru
Asha Rao	Chennai
Chitra Nair	Chennai

NULLs sort differently across databases, so state where you want them.

Always break ties

`ORDER BY amount DESC LIMIT 10` can return a different tenth row each time if amounts tie. Add a unique column as the final sort key.

Pagination: OFFSET vs keyset

```sql
-- OFFSET: simple, but the database still reads the skipped rows
SELECT id, amount FROM orders
ORDER BY id
LIMIT 3 OFFSET 3;
```

RESULT · 3 ROWS

id	amount
104	1200.00
105	8300.00
106	2400.00

```sql
-- Keyset: continue from the last row you saw. Stays fast on deep pages.
SELECT id, amount FROM orders
WHERE id > 106
ORDER BY id
LIMIT 3;
```

RESULT · 3 ROWS

id	amount
107	22500.00
108	1800.00
109	6500.00

Keyset pagination also avoids skipped or repeated rows when new data arrives between page loads. The trade-off is that you can't jump straight to page 50.

CHAPTER 11

Beyond SELECT

Create tables, change data, and undo mistakes safely.

Queries read data. The rest of SQL defines structures, changes rows, and controls access. You will use these less often, but mistakes here cost more.

DDL: define the structure

```
CREATE TABLE coupons (
  code       TEXT PRIMARY KEY,
  percent    INT NOT NULL CHECK (percent BETWEEN 1 AND 90),
  product_id INT REFERENCES products(id),
  expires_on DATE
);

ALTER TABLE coupons ADD COLUMN active BOOLEAN NOT NULL DEFAULT TRUE;
CREATE INDEX idx_coupons_product ON coupons (product_id);
CREATE VIEW completed_orders AS
  SELECT * FROM orders WHERE status = 'completed';
```

> **Let constraints guard your data**
> `NOT NULL`, `UNIQUE`, `CHECK`, and foreign keys stop bad data at the door, whichever application writes it.

DML: change the rows

`RETURNING` shows exactly what changed. It works in PostgreSQL; other databases have their own forms.

```
UPDATE products
SET price = price * 1.10
WHERE category = 'Displays'
RETURNING id, name, price;
```

RESULT · 1 ROW

id	name	price
3	27in Monitor	19800.00

```
INSERT INTO products (id, name, category, price)
VALUES (7, 'Desk Mat', 'Accessories', 900)
RETURNING id, name;
```

RESULT · 1 ROW

id	name
7	Desk Mat

Trap · An UPDATE or DELETE without WHERE hits every row
Write the `SELECT` with the same `WHERE` first, check the rows, then change the first line.

TCL: make changes all-or-nothing

```
BEGIN;
UPDATE orders SET status = 'cancelled' WHERE customer_id = 1;
SELECT COUNT(*) FROM orders WHERE status = 'completed';  -- 9
ROLLBACK;                                  -- changed your mind
SELECT COUNT(*) FROM orders WHERE status = 'completed';  -- 13
```

Use `COMMIT` instead of `ROLLBACK` to keep the changes. Wrap related writes in one transaction, such as debiting one account and crediting another.

DCL: control who can do what

```
CREATE ROLE analyst;
GRANT SELECT ON orders, customers TO analyst;
REVOKE SELECT ON customers FROM analyst;
```

Least privilege
Give people and applications only the access they need. A reporting user rarely needs more than `SELECT`.

`DELETE`
Removes chosen rows. Can be filtered.

`TRUNCATE`
Quickly removes all rows. Keeps the table.

`DROP`
Removes the table itself, with its data.

Data types worth knowing

Family	Common types	Use it for
Numeric	`INT`, `BIGINT`, `NUMERIC(10,2)`	Counts and IDs. Use `NUMERIC` for money, never `FLOAT`.
String	`TEXT`, `VARCHAR(n)`	Names, emails, codes
Date & time	`DATE`, `TIMESTAMP`, `TIMESTAMPTZ`	Events. Prefer a time-zone-aware type for real-world moments.
Boolean	`BOOLEAN`	Yes/no flags
JSON	`JSONB`, `JSON`	Flexible attributes. Don't hide columns you filter on.

RECIPES

8 queries you will reuse

Copy, rename the columns, ship.

1 Top N per group

Highest-paid two people in each department. A staple of SQL interviews.

```sql
WITH ranked AS (
  SELECT name, dept, salary,
         ROW_NUMBER() OVER (PARTITION BY dept
                            ORDER BY salary DESC, id) AS rn
  FROM employees
)
SELECT dept, name, salary
FROM ranked
WHERE rn <= 2
ORDER BY dept, rn;
```

RESULT · 5 ROWS

dept	name	salary
Engineering	Meera Iyer	240000
Engineering	Arjun Das	180000
Sales	Uma Pillai	160000
Sales	Sana Sheikh	150000
Support	Vikram Shah	80000

2 Percentage of total

Each category's share of completed revenue.

```sql
SELECT p.category,
       SUM(i.qty * p.price) AS revenue,
       ROUND(100.0 * SUM(i.qty * p.price)
             / SUM(SUM(i.qty * p.price)) OVER (), 1) AS pct
FROM order_items i
JOIN products p ON p.id = i.product_id
JOIN orders o   ON o.id = i.order_id
WHERE o.status = 'completed'
GROUP BY p.category
ORDER BY revenue DESC;
```

RESULT · 3 ROWS

category	revenue	pct
Displays	54000.00	49.5
Accessories	36800.00	33.7
Peripherals	18300.00	16.8

SUM(SUM(...)) OVER () looks odd, but reads simply: the window total of the grouped sums.

3 Deduplicate, keeping the latest

Normalize the key, number the rows, and keep the newest in each group. Run this SELECT first, so you can see what would survive.

```
WITH numbered AS (
  SELECT id, LOWER(TRIM(email)) AS email, source, created_at,
         ROW_NUMBER() OVER (PARTITION BY LOWER(TRIM(email))
                            ORDER BY created_at DESC, id DESC) AS rn
  FROM signups_raw
)
SELECT id, email, source, created_at
FROM numbered
WHERE rn = 1
ORDER BY id;
```

RESULT · 4 ROWS

id	email	source	created_at
2	asha@example.com	mobile	2025-01-14 18:30:00
3	ben@example.com	web	2025-01-20 11:15:00
5	dev@example.com	web	2025-02-17 08:06:00
6	esha@example.com	mobile	2025-03-05 20:40:00

4 Fill gaps in a time series

Months with no sales should show 0, not vanish from your chart.

```
WITH months AS (
  SELECT generate_series(DATE '2025-05-01', DATE '2025-07-01',
                         INTERVAL '1 month')::date AS month
)
SELECT m.month, COALESCE(SUM(o.amount), 0) AS revenue
FROM months m
LEFT JOIN orders o
       ON o.order_date >= m.month
      AND o.order_date <  m.month + INTERVAL '1 month'
      AND o.status = 'completed'
GROUP BY m.month
ORDER BY m.month;
```

RESULT · 3 ROWS

month	revenue
2025-05-01	1800.00
2025-06-01	0
2025-07-01	18000.00

5 First and most recent event

Each customer's first and latest order, plus the gap between them.

```sql
SELECT customer_id,
       MIN(order_date) AS first_order,
       MAX(order_date) AS last_order,
       MAX(order_date) - MIN(order_date) AS active_days
FROM orders
WHERE customer_id IS NOT NULL AND status = 'completed'
GROUP BY customer_id
ORDER BY active_days DESC;
```

RESULT · 6 ROWS

customer_id	first_order	last_order	active_days
1	2025-01-15	2025-07-04	170
2	2025-01-28	2025-05-06	98
5	2025-03-11	2025-04-28	48
3	2025-04-02	2025-04-02	0
4	2025-02-25	2025-02-25	0
6	2025-04-15	2025-04-15	0

6 Repeat purchase rate

What share of buyers came back?

```sql
WITH per_customer AS (
  SELECT customer_id, COUNT(*) AS orders
  FROM orders
  WHERE status = 'completed' AND customer_id IS NOT NULL
  GROUP BY customer_id
)
SELECT COUNT(*) AS buyers,
       COUNT(*) FILTER (WHERE orders > 1) AS repeat_buyers,
       ROUND(100.0 * COUNT(*) FILTER (WHERE orders > 1) / COUNT(*), 1)
         AS repeat_pct
FROM per_customer;
```

RESULT · 1 ROW

buyers	repeat_buyers	repeat_pct
6	3	50.0

7 Monthly cohorts

Group customers by the month they first bought, then count how many bought again in each later month.

```sql
WITH firsts AS (
  SELECT customer_id,
         DATE_TRUNC('month', MIN(order_date))::date AS cohort
  FROM orders
  WHERE status = 'completed' AND customer_id IS NOT NULL
  GROUP BY customer_id
)
SELECT f.cohort,
       DATE_TRUNC('month', o.order_date)::date AS active_month,
       COUNT(DISTINCT o.customer_id) AS customers
FROM firsts f
JOIN orders o ON o.customer_id = f.customer_id
WHERE o.status = 'completed'
GROUP BY 1, 2
ORDER BY 1, 2
LIMIT 6;
```

RESULT · 6 ROWS

cohort	active_month	customers
2025-01-01	2025-01-01	2
2025-01-01	2025-02-01	1
2025-01-01	2025-03-01	2
2025-01-01	2025-05-01	1
2025-01-01	2025-07-01	1
2025-02-01	2025-02-01	1

8 Median, not just average

One big order can drag an average upward. The median resists it.

```sql
SELECT ROUND(AVG(amount), 2) AS mean,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) AS median
FROM orders
WHERE status = 'completed';
```

RESULT · 1 ROW

mean	median
8392.31	5700

Trap · Deleting duplicates
Check the SELECT first, and run the delete inside a transaction. This PostgreSQL form removes every row that isn't the newest for its key:

```
DELETE FROM signups_raw
WHERE id IN (
  SELECT id FROM (
    SELECT id, ROW_NUMBER() OVER (PARTITION BY LOWER(TRIM(email))
                                  ORDER BY created_at DESC, id DESC) AS rn
    FROM signups_raw) t
  WHERE rn > 1);
```

CHECKLIST

10 silent bugs

None of them raises an error. Check before you ship.

- [ ] **Filtering the right table of a LEFT JOIN in WHERE**
  It becomes an INNER JOIN. Move the condition to ON.
- [ ] **NOT IN with a nullable column**
  One NULL returns zero rows. Use NOT EXISTS.
- [ ] **BETWEEN on timestamps**
  It misses nearly all of the last day. Use >= and <.
- [ ] **Unparenthesized AND / OR**
  AND binds first. Add parentheses.
- [ ] **Integer division**
  In PostgreSQL and SQL Server, 1 / 2 is 0. Write 1.0 * a / b.
- [ ] **Joining before aggregating**
  One-to-many joins repeat values and inflate totals.
- [ ] `col = NULL` **or** `col <> 'x'`
  NULL rows silently drop out. Use IS NULL or IS DISTINCT FROM.
- [ ] **LIMIT without a unique sort key**
  Ties make results change between runs.
- [ ] **Running totals over tied values**
  The default RANGE frame merges ties. Use ROWS or a tie-breaker.
- [ ] **SELECT DISTINCT as a bug fix**
  It hides fan-out rather than fixing it. Find the join.

A slow query? Work down this list

1. Read the plan with `EXPLAIN ANALYZE`, rather than guessing.
2. Check for an accidental fan-out, which often shows as far more rows than expected.
3. Index the columns you filter and join on, starting with the most selective.
4. Keep indexed columns bare: use a range instead of a function on the column.
5. Select only the columns you need, and aggregate before joining.
6. Replace deep `OFFSET` with keyset pagination.

PRACTICE

12 exercises

Write each one against the sample database before checking the answer.

Level 1 · Warm-up

1. How many completed orders are there, and what is their total amount?
2. List each product category with its number of products, largest first.
3. Which customers have no email on file?
4. Which departments pay an average salary above 120,000?

Level 2 · Everyday work

5. Show total completed spend per customer name, including customers who spent nothing.
6. Which products have never been sold?
7. Which employees earn more than their manager?
8. Which city has the highest completed revenue? Ignore customers with no city.

Level 3 · Interview ready

9. Find the second-highest distinct salary.
10. For each customer's completed orders, show the days since their previous order. Limit it to the first 4 rows for customers 1 and 2.
11. Find each customer's largest completed order. Return the customer ID, order ID, and amount.
12. Which pairs of products were bought in the same order? Count each pair once.

ANSWERS

Answers

Your query may look different. If the result matches, it is probably right too.

1 How many completed orders are there, and what is their total amount?

```sql
SELECT COUNT(*) AS orders, SUM(amount) AS total
FROM orders WHERE status = 'completed';
```

RESULT · 1 ROW

orders	total
13	109100.00

2 List each product category with its number of products, largest first.

```sql
SELECT category, COUNT(*) AS products
FROM products GROUP BY category
ORDER BY products DESC, category;
```

RESULT · 3 ROWS

category	products
Peripherals	3
Accessories	2
Displays	1

3 Which customers have no email on file?

```sql
SELECT name FROM customers WHERE email IS NULL;
```

RESULT · 1 ROW

name
Chitra Nair

4 Which departments pay an average salary above 120,000?

```sql
SELECT dept, ROUND(AVG(salary)) AS avg_salary
FROM employees GROUP BY dept
HAVING AVG(salary) > 120000 ORDER BY avg_salary DESC;
```

RESULT · 2 ROWS

dept	avg_salary
Engineering	180000
Sales	135000

5 Show total completed spend per customer name, including customers who spent nothing.

```sql
SELECT c.name, COALESCE(SUM(o.amount), 0) AS spend
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.id AND o.status = 'completed'
GROUP BY c.name ORDER BY spend DESC, c.name;
```

RESULT · 7 ROWS

name	spend
Esha Khan	35500.00
Asha Rao	33800.00
Ben Thomas	22200.00
Dev Patel	8300.00
Chitra Nair	4500.00
Farid Ali	3000.00
Gita Menon	0

6 Which products have never been sold?

```sql
SELECT p.name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items i
                  WHERE i.product_id = p.id);
```

RESULT · 1 ROW

name
Webcam

7 Which employees earn more than their manager?

```sql
SELECT e.name FROM employees e
JOIN employees m ON m.id = e.manager_id
WHERE e.salary > m.salary;
```

RESULT · 1 ROW

name
Uma Pillai

8 Which city has the highest completed revenue? Ignore customers with no city.

```sql
SELECT c.city, SUM(o.amount) AS revenue
FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.status = 'completed' AND c.city IS NOT NULL
GROUP BY c.city ORDER BY revenue DESC LIMIT 1;
```

RESULT · 1 ROW

city	revenue
Bengaluru	57700.00

9 Find the second-highest distinct salary.

```sql
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS r
  FROM employees) t
WHERE r = 2;
```

RESULT · 1 ROW

salary
180000

10 For each customer's completed orders, show the days since their previous order. Limit it to the first 4 rows for customers 1 and 2.

```
SELECT customer_id, order_date,
  order_date - LAG(order_date) OVER (
    PARTITION BY customer_id ORDER BY order_date) AS days_since_prev
FROM orders
WHERE status = 'completed' AND customer_id IN (1, 2)
ORDER BY customer_id, order_date LIMIT 4;
```

RESULT · 4 ROWS

customer_id	order_date	days_since_prev
1	2025-01-15	*NULL*
1	2025-02-10	26
1	2025-03-30	48
1	2025-07-04	96

11 Find each customer's largest completed order. Return the customer ID, order ID, and amount.

```
SELECT customer_id, id, amount FROM (
  SELECT customer_id, id, amount,
    ROW_NUMBER() OVER (PARTITION BY customer_id
                       ORDER BY amount DESC, id) AS rn
  FROM orders
  WHERE status = 'completed' AND customer_id IS NOT NULL) t
WHERE rn = 1 ORDER BY customer_id;
```

RESULT · 6 ROWS

customer_id	id	amount
1	115	18000.00
2	102	18000.00
3	110	4500.00
4	105	8300.00
5	107	22500.00
6	112	3000.00

12 Which pairs of products were bought in the same order? Count each pair once.

```
SELECT a.name AS product_a, b.name AS product_b, COUNT(*) AS times
FROM order_items x
JOIN order_items y
  ON y.order_id = x.order_id AND y.product_id > x.product_id
JOIN products a ON a.id = x.product_id
JOIN products b ON b.id = y.product_id
GROUP BY 1, 2 ORDER BY times DESC, 1, 2;
```

RESULT · 4 ROWS

product_a	product_b	times
Mechanical Keyboard	27in Monitor	1
Mechanical Keyboard	Wireless Mouse	1
USB-C Dock	Laptop Stand	1
Wireless Mouse	Laptop Stand	1

INTERVIEW

8 interview questions

Short answers you can say out loud.

What is the difference between WHERE and HAVING?

WHERE filters rows before grouping. HAVING filters groups after aggregation, so it can use aggregates such as SUM.

RANK, DENSE_RANK or ROW_NUMBER?

All three number rows in order. On ties, ROW_NUMBER still gives unique numbers, RANK repeats and leaves a gap, and DENSE_RANK repeats with no gap. Use ROW_NUMBER for “exactly N rows”, DENSE_RANK for “the Nth highest value”.

Why might NOT IN return no rows?

If the subquery returns any NULL, the condition can never be true. Use NOT EXISTS.

UNION vs UNION ALL?

UNION removes duplicates, at extra cost. UNION ALL keeps every row and is the safer default.

COUNT(*) vs COUNT(col)?

COUNT(*) counts rows. COUNT(col) counts only rows where col is not NULL.

Why did my total go up after a join?

Fan-out: one row matched many, repeating its values. Aggregate the many side first, or join at the right grain.

When is a CTE better than a subquery?

When a step is reused, or the logic reads better top to bottom, and for recursion, which needs a CTE. Performance is usually similar, but it varies by engine, so check the plan.

How would you speed up a slow query?

Read the execution plan first. Then filter earlier, index the columns used in filters and joins, avoid functions on indexed columns, select only the columns you need, and check for accidental fan-out.

CHEAT SHEET

One-page cheat sheet

Aggregate

```
SELECT k, SUM(x) AS total
FROM t
WHERE cond
GROUP BY k
HAVING SUM(x) > 100;
```

Join (keep the left)

```
SELECT a.*, b.y
FROM a
LEFT JOIN b ON b.a_id = a.id
  AND b.flag = 1;
```

No match

```
SELECT a.* FROM a
WHERE NOT EXISTS (
  SELECT 1 FROM b
  WHERE b.a_id = a.id);
```

CTE

```
WITH s AS (
  SELECT k, SUM(x) AS x
  FROM t GROUP BY k
)
SELECT * FROM s WHERE x > 10;
```

Top N per group

```
SELECT * FROM (
  SELECT t.*, ROW_NUMBER() OVER (
    PARTITION BY k ORDER BY x DESC
  ) AS rn FROM t
) r WHERE rn <= 3;
```

Previous row, running total

```
LAG(x) OVER (ORDER BY d)
SUM(x) OVER (ORDER BY d, id)
```

Share of group

```
100.0 * x
  / SUM(x) OVER (PARTITION BY k)
```

Pivot

```
SUM(CASE WHEN s = 'a'
         THEN 1 ELSE 0 END) AS a
```

NULL handling

```
COALESCE(x, 0)
x / NULLIF(y, 0)
x IS NULL
```

Date range

```
d >= '2025-01-01'
AND d < '2025-02-01'
```

Safe write

```
BEGIN;
UPDATE t SET x = 1 WHERE id = 7;
-- check it, then:
COMMIT;  -- or ROLLBACK;
```

Define

```
CREATE TABLE t (
  id INT PRIMARY KEY,
  x  INT NOT NULL CHECK (x > 0)
);
```

Logical order of execution

FROM → JOIN → WHERE → GROUP BY → HAVING → window functions → SELECT → DISTINCT → ORDER BY → LIMIT

DIALECTS

Moving between databases

Examples in this guide run on PostgreSQL. These are the differences you will hit first.

Task	PostgreSQL	MySQL 8	SQL Server	Oracle
First N rows	`LIMIT n`	`LIMIT n`	`TOP (n)`	FETCH FIRST n ROWS ONLY
Floor to month	`DATE_TRUNC('month', d)`	`DATE_FORMAT(d, '%Y-%m-01')`	`DATETRUNC(month, d)` (2022+)	`TRUNC(d, 'MM')`
Add 7 days	`d + INTERVAL '7 days'`	`DATE_ADD(d, INTERVAL 7 DAY)`	`DATEADD(day, 7, d)`	`d + 7`
Days between	`d2 - d1`	`DATEDIFF(d2, d1)`	`DATEDIFF(day, d1, d2)`	`d2 - d1`
Join strings	`a || b`	`CONCAT(a, b)`	`CONCAT(a, b)`	`a || b`
Subtract sets	`EXCEPT`	`EXCEPT (8.0.31+)`	`EXCEPT`	`MINUS`
FULL JOIN	Yes	No; emulate with UNION	Yes	Yes
Case-insensitive match	`ILIKE`	Usually LIKE (depends on collation)	Usually LIKE (depends on collation)	`LOWER(x) LIKE`

Syntax can vary by version and configuration. Check your database's documentation when in doubt.

APPENDIX

Sample database script

Paste into any PostgreSQL database to run every example. It drops and recreates only these six tables.

```
DROP TABLE IF EXISTS order_items, orders, products, customers, employees, signups_raw;
CREATE TABLE customers (id INT PRIMARY KEY, name TEXT NOT NULL, email TEXT, city TEXT, signup_date DATE NOT NULL);
CREATE TABLE products (id INT PRIMARY KEY, name TEXT NOT NULL, category TEXT NOT NULL, price NUMERIC(10,2) NOT
NULL);
CREATE TABLE orders (id INT PRIMARY KEY, customer_id INT REFERENCES customers(id), order_date DATE NOT NULL,
status TEXT NOT NULL, amount NUMERIC(10,2));
CREATE TABLE order_items (order_id INT REFERENCES orders(id), product_id INT REFERENCES products(id), qty INT NOT
NULL, PRIMARY KEY (order_id, product_id));
CREATE TABLE employees (id INT PRIMARY KEY, name TEXT NOT NULL, dept TEXT NOT NULL, salary INT NOT NULL,
manager_id INT REFERENCES employees(id));
CREATE TABLE signups_raw (id INT PRIMARY KEY, email TEXT NOT NULL, source TEXT, created_at TIMESTAMP NOT NULL);
INSERT INTO customers VALUES
(1,'Asha Rao','asha@example.com','Chennai','2025-01-12'),
(2,'Ben Thomas','ben@example.com','Bengaluru','2025-01-20'),
(3,'Chitra Nair',NULL,'Chennai','2025-02-03'),
(4,'Dev Patel','dev@example.com','Mumbai','2025-02-17'),
(5,'Esha Khan','esha@example.com','Bengaluru','2025-03-05'),
(6,'Farid Ali','farid@example.com',NULL,'2025-03-09'),
(7,'Gita Menon','gita@example.com','Chennai','2025-04-21');
INSERT INTO products VALUES
(1,'Mechanical Keyboard','Peripherals',4500),(2,'Wireless Mouse','Peripherals',1200),
(3,'27in Monitor','Displays',18000),(4,'USB-C Dock','Accessories',6500),
(5,'Laptop Stand','Accessories',1800),(6,'Webcam','Peripherals',3200);
INSERT INTO orders (id,customer_id,order_date,status) VALUES
(101,1,'2025-01-15','completed'),(102,2,'2025-01-28','completed'),(103,1,'2025-02-10','completed'),
(104,3,'2025-02-14','cancelled'),(105,4,'2025-02-25','completed'),(106,2,'2025-03-03','completed'),
(107,5,'2025-03-11','completed'),(108,NULL,'2025-03-18','completed'),(109,1,'2025-03-30','completed'),
(110,3,'2025-04-02','completed'),(111,4,'2025-04-09','pending'),(112,6,'2025-04-15','completed'),
(113,5,'2025-04-28','completed'),(114,2,'2025-05-06','completed'),(115,1,'2025-07-04','completed');
INSERT INTO order_items VALUES
(101,1,1),(101,2,1),(102,3,1),(103,5,2),(104,2,1),(105,4,1),(105,5,1),(106,2,2),(107,3,1),(107,1,1),
(108,5,1),(109,4,1),(110,1,1),(111,3,1),(112,2,1),(112,5,1),(113,4,2),(114,5,1),(115,3,1);
UPDATE orders o SET amount = (SELECT SUM(i.qty*p.price) FROM order_items i JOIN products p ON p.id=i.product_id
WHERE i.order_id=o.id);
INSERT INTO employees VALUES
(1,'Meera Iyer','Engineering',240000,NULL),(2,'Arjun Das','Engineering',180000,1),(3,'Neha
Joshi','Engineering',180000,1),
(4,'Ravi Kumar','Engineering',120000,2),(5,'Sana Sheikh','Sales',150000,1),(6,'Tom Mathew','Sales',95000,5),
(7,'Uma Pillai','Sales',160000,5),(8,'Vikram Shah','Support',80000,5);
INSERT INTO signups_raw VALUES
(1,'asha@example.com','web','2025-01-12 09:00'),(2,'ASHA@example.com ','mobile','2025-01-14 18:30'),
(3,'ben@example.com','web','2025-01-20 11:15'),(4,'dev@example.com','ads','2025-02-17 08:05'),
(5,'dev@example.com','web','2025-02-17 08:06'),(6,'esha@example.com','mobile','2025-03-05 20:40');
```

AUTHOR

About the author

Karthikeyan T is a Senior AI Engineer, LLM Engineer, and SaaS Architect with 20+ years of engineering experience, building production AI and multi-tenant SaaS systems.

His work spans RAG pipelines, AI agents, security tooling, and full-stack platforms on PostgreSQL. Past roles include Virtusa, Buckman, and AURISS Technologies.

Building something that needs this kind of engineering?

Get in touch about AI systems, SaaS architecture, or data-heavy applications.

Email	**tkarthikeyan@gmail.com**
Portfolio	**carthworks.vercel.app**
LinkedIn	**linkedin.com/in/carthworks**
GitHub	**github.com/carthworks**

© 2026 Karthikeyan T. All rights reserved. This guide is licensed for the buyer's personal use. Please don't resell or redistribute it. Code examples may be used freely in your own work. Sample data is fictional. Examples were run on PostgreSQL 16.